

# Emotion-AI Programming Language (EAPL)

## Documentation

**Overview:** The **Emotion-AI Programming Language (EAPL)** is a compact, DSL-like notation that encodes conversational context (roles, domain) and emotional tone in a few characters. Its goal is to provide AI agents with rich situational cues and sentiment in a micro-format – even **“shorter than a Twitter handle”** – so that LLMs can respond with appropriate emotion and role-awareness. In EAPL, each string specifies the *domain* (e.g. hotel), the *agent role* (e.g. concierge), the *other party's role* (e.g. guest), and an *emotional tone* (e.g. angry, excited, professional). By exposing this context to AI models, we enable more intuitive, empathetic interactions. For example, a string like `hotel--concierge--guest--angry` tells the agent: **you are a hotel concierge speaking to an angry guest**. Embedding such cues can dramatically improve user satisfaction in customer-service bots <sup>1</sup> <sup>2</sup>.

EAPL is designed to integrate with modern AI toolchains. In practice, EAPL strings are passed via a JSON payload (see **JSON API** below) and delivered to LLMs through a context protocol. The emerging *Model Context Protocol (MCP)* provides a standard “USB-C port for AI” to inject this context <sup>3</sup>. We will provide an SDK so developers can easily parse EAPL strings, build prompts, and attach emotional context to agents. The end result is a **killer blueprint**: a minimal-context specification that becomes the global standard for emotion-aware AI, enabling role-based agents that are *smart, empathetic, and highly effective*.

## Key Concepts

- **Domain/Context:** The general setting of the conversation (e.g. `hotel`, `restaurant`, `business`). This anchors the scenario.
- **Agent Role:** The AI's persona or job function (e.g. `concierge`, `manager`, `customer-experience-agent`). These roles come from everyday service or business contexts. (AI agents, like humans, operate in distinct roles <sup>4</sup>.)
- **Customer Role:** The human's identity or relation (e.g. `guest`, `attorney`, `hotel-employee`, `guestofguest`). This distinguishes e.g. a hotel guest vs. an employee or visitor.
- **Emotion/Tone:** A final tag describing the speaker's emotional state or style (e.g. `angry`, `excited`, `professional`, `confused`). This tells the AI *how* to speak. In EAPL, “emotion” may be an affect label or a conversational tone. For example, `professional` means a formal, helpful tone, while `angry` or `excited` specify mood. Providing emotion helps the AI adapt its response empathetically <sup>2</sup> <sup>5</sup>.

These four elements (domain, agent, customer, emotion) are concatenated with double hyphens `--`. For instance, the template

```
DOMAIN--AGENT_ROLE--CUSTOMER_ROLE--EMOTION
```

yields examples like:

```
hotel--concierge--guest--angry
hotel--manager--hotel-employee--confused
hotel--customer-experience-agent--guest--excited
```

In each case the AI knows its setting (hotel), who it is (concierge, manager, etc.), who it speaks to (guest, employee, etc.), and what emotional tone to adopt (angry, excited, confused). This compact syntax encodes all *primary roles* and context in one line <sup>6</sup> <sup>7</sup> .

- **Primary Roles (Example):** In a hospitality context, the receiver (service provider) might be a *business* and the customer a *guest*. For instance:

```
{{receiver:business}}--concierge--{{customer:guest}}--{{emotion}}
```

Here “receiver: business” and “customer: guest” are variable tags indicating role categories. In a hotel scenario, the concierge (agent) serves a guest (customer) with a specified emotion. By contrast, in a corporate context you might use `business--customer-experience-agent--client--professional` . These placeholders show how EAPL can generalize to any role type.

**Emotional AI Goal:** Emotional AI agents aim to interpret and respond to human emotions, creating *more intuitive, context-aware interactions between humans and machines* <sup>1</sup> <sup>2</sup> . EAPL directly supports this by encoding the needed context. W3C’s *Emotion Markup Language* (EmotionML) similarly recognized that “representing the emotional states of a user or the emotional states to be simulated by a user interface requires a suitable representation format” <sup>6</sup> . EAPL is a concise evolution of that idea, optimized for AI agents and minimal bandwidth.

## Syntax and Structure

EAPL strings follow a strict grammar:

```
STRUCTURE := [PREFIX/][Domain]--[Agent]--[Customer]--[Emotion]
```

- Fields are separated by double hyphens `--` .
- **Domain** is a short keyword (no spaces) like `hotel` , `shop` , `flight` , etc. This may also appear as a path prefix (e.g. `Prompts/hotel` ) to organize scenarios.
- **Agent** is the AI’s role or title (e.g. `concierge` , `manager` , `customer-service-agent` ).
- **Customer** is the human’s role (e.g. `guest` , `attorney` , `hotel-employee` ). Hyphenated terms like `guest-of-guest` or `customer-experience-agent` are allowed.
- **Emotion** is an affect tag or style (e.g. `angry` , `excited` , `professional` , `aggravated` ). This word guides the agent’s tone.

All parts are ASCII strings without spaces. For example:

- `hotel--concierge--guest--angry` – Agent: concierge, Customer: guest, Tone: angry (in a hotel setting).
- `hotel--manager--hotel-employee--confused` – Agent: manager, Customer: employee, Tone: confused.
- `Prompts/hotel--concierge--guest-attorney--professional` – A professional tone for a concierge speaking to a guest who is also an attorney (here `guest-attorney` is hyphenated in the Customer field).

This syntax **maximizes brevity**. A single EAPL line (often under 30 characters) fully specifies the dialogue context. By encoding roles and emotional state explicitly, we avoid verbose system prompts. As Dr. Lisa Barrett notes, *“context is key to understanding emotion”* <sup>5</sup> ; EAPL makes context explicit.

## JSON API Structure

EAPL is designed to be transmitted via a JSON REST API. For example, an AI application could expose an endpoint like `POST /api/emotion-context` that consumes EAPL context. The JSON request might look like:

```
POST /api/emotion-context
Content-Type: application/json

{
  "domain": "hotel",
  "roles": {
    "agent": "concierge",
    "customer": "guest"
  },
  "emotion": "angry"
}
```

### Fields explained:

- `"domain"` – matches the EAPL Domain (e.g. `"hotel"`).
- `"roles"` – an object with `"agent"` and `"customer"` keys (e.g. agent: `"concierge"`, customer: `"guest"`).
- `"emotion"` – the emotional tone string (e.g. `"angry"`).

The API should use HTTP with `application/json` (UTF-8) as the data format <sup>8</sup>. This follows standard RESTful design (using nouns, HTTP verbs, JSON payloads) <sup>8</sup>. A sample **curl** could be:

```
curl -X POST https://example.com/api/emotion-context \
-H "Content-Type: application/json" \
```

```
-d '{"domain":"hotel","roles":  
{"agent":"concierge","customer":"guest"},"emotion":"angry"}'
```

Upon receiving this JSON, the backend can reconstruct the EAPL context string ( `hotel--concierge--guest--angry` ) or process it directly. For instance, an AI agent framework might pass this JSON into the prompt. The key is that every piece of context is structured and labelled, not buried in natural language. The **Emotion-AI SDK** (below) will include client libraries to build and send this JSON easily.

## Integration with AI Agents

EAPL is meant to feed context into AI models. The recommended approach is via the **Model Context Protocol (MCP)**, which standardizes how applications provide context to LLMs <sup>3</sup>. Think of MCP like a “USB-C port” for AI: any context (including EAPL data) can be plugged in through MCP <sup>3</sup>. Our blueprint will include an MCP server or tool that reads EAPL strings and exposes them to the model. For example, an LLM agent can use an MCP tool call to fetch the current `domain`, `roles`, and `emotion` from a JSON object. This ensures the AI model always has the latest context as it generates responses.

In practice, you might configure an MCP JSON tool in your agent as:

```
tools:  
- type: mcp  
  endpoint: "https://myapp.com/api/emotion-context"  
  method: "POST"
```

The model then automatically receives this context object before replying. By embedding EAPL fields, the LLM can “know” that it is role-playing a concierge and speaking to a frustrated guest. This follows MCP best practices for context injection <sup>3</sup> <sup>8</sup>.

## Role-Based Agents

EAPL also ties into the emerging practice of **role-based AI agents**. Future AI systems are envisioned to have well-defined roles (and corresponding permissions) in multi-agent workflows <sup>4</sup>. Our language explicitly names the agent’s role in each context string, aligning with this vision. For example, a hotel’s *general-manager* and *customer-experience-agent* are distinct roles. By making the role part of the context (via EAPL), we ensure the AI behaves consistently with that persona. As HabiliAI docs note, agents will be “assigned distinct roles” and operate within defined boundaries <sup>4</sup>. EAPL provides a syntactic foundation for that – every prompt starts with the agent’s role and mission.

## SDK & Developer Tools

To make EAPL practical, we will release an **Emotion-AI SDK** for developers. Key features include:

- **Parser/Validator:** Functions to parse EAPL strings and verify they follow the grammar (correct fields, no missing parts).

- **JSON Builder:** Helpers to construct the JSON payload from given context values. (For example, `buildContext({domain:"hotel", agent:"concierge", customer:"guest", emotion:"angry"})`.)
- **Role Enumerations:** Predefined lists or enums of common domains, roles, and emotions, so developers can autocomplete (e.g. `Roles.CONCIERGE`, `Emotion.ANGRY`).
- **Prompts & Templates:** Utilities to map EAPL context into natural-language prompts or system instructions for LLMs (e.g. "You are a *hotel concierge* talking to an *angry guest*..."). This bridges EAPL data with the LLM's input format.
- **MCP Integration:** Out-of-the-box support for injecting EAPL into agents via MCP. For example, a function to register an EAPL MCP endpoint, or example code to add an `EmotionContextTool`.
- **Language Support:** Initially, the SDK will be available in popular languages (e.g. Python, TypeScript) so AI developers can easily integrate EAPL into their stacks (analogous to other AI SDKs like Vercel's <sup>9</sup>).

The SDK's documentation will include code samples. For instance, a Python snippet:

```
from emotionai_sdk import EmotionContext

ctx = EmotionContext(domain="hotel", agent="concierge", customer="guest",
emotion="angry")
json_payload = ctx.to_json()
# Send json_payload to the AI backend or MCP tool
```

By providing these developer utilities, we make adopting EAPL straightforward. The concise nature of the language means agents need to parse just a few tokens, not entire sentences, to grasp context. This efficiency helps agents "operate as a self-contained system" that is both *autonomous* and *context-aware*, echoing principles of agent design <sup>10</sup> <sup>11</sup>.

## Example: Hotel Concierge Scenario

Consider a hotel chatbot. We can illustrate EAPL with a sequence of contexts:

- **Context:** `hotel--concierge--guest--professional`  
*Interpretation:* A hotel concierge is speaking to a guest with a professional, courteous tone. (This might be the default welcome mode.)
- **Context:** `hotel--concierge--guest--attorney--professional`  
*Interpretation:* The guest is also an attorney; the concierge remains professional and informative.
- **Context:** `hotel--concierge--guest--angry`  
*Interpretation:* The guest is angry (perhaps due to a booking issue). The concierge should respond empathically, de-escalating frustration. (Empathy in a response boosts user satisfaction <sup>2</sup>.)
- **Context:** `hotel--general-manager--guest--aggravated`  
*Interpretation:* The guest has complained up to the general manager, who now speaks in a concerned but authoritative manner.
- **Context:** `hotel--manager--hotel-employee--confused`  
*Interpretation:* A manager is addressing a confused hotel employee, using clarifying and patient tone.

In each case, the agent's behavior is guided by the tiny EAPL string. For example, when the LLM sees `hotel--concierge--guest--angry`, it might generate: *"I understand you're upset. Let me help resolve this right away,"* showing empathy. These nuanced responses follow from EAPL's cues – *detecting frustration and offering support* <sup>2</sup> .

## Benefits and Standards

- **Human-like Interaction:** Encoding emotion allows AI to mirror human empathy. Studies show that empathetic AI (e.g. recognizing frustration and responding kindly) dramatically improves perceived user satisfaction <sup>2</sup> <sup>12</sup> . EAPL ensures AIs have the necessary emotional cue.
- **Conciseness:** A micro-language like EAPL avoids bulky prompts. Rather than a paragraph describing the situation, a few tokens suffice. This is crucial for efficiency in multi-agent systems where context may update rapidly.
- **Standardization:** By defining a clear schema, EAPL can become a universal protocol for emotional context. Just as MCP standardizes technical context <sup>3</sup> , EAPL aims to standardize affective context.
- **Developer-Friendly:** With an SDK, EAPL integrates smoothly into LLM pipelines. AI developers can focus on building the application, confident that the language handles emotional context systematically.

## Conclusion

The **Emotion-AI Programming Language** is a revolutionary blueprint for infusing AI/human interactions with emotion and role-awareness in the briefest form. It complements existing standards (MCP for context, REST/JSON for data exchange <sup>8</sup> <sup>3</sup> ) while introducing a specialized syntax for affect. By making context explicit, EAPL helps create agents that are *empathetic*, *focused*, and *effective* – truly "context-aware" assistants. As Dr. Rosalind Picard of MIT notes, this integration of emotion isn't just technical progress – it's a step toward *more effective communication between humans and machines* <sup>13</sup> . EAPL is the lightweight yet powerful language that will drive that future.

**Sources:** The design draws on standards in affective computing and AI protocols. Emotional AI agent design is discussed in Milvus documentation <sup>1</sup> and SmythOS blogs <sup>12</sup> <sup>2</sup> . REST API best-practices (JSON over HTTP) follow guidelines <sup>8</sup> . The Model Context Protocol (MCP) standard informs our integration approach <sup>3</sup> . All are cited above.

---

1 What are emotional AI agents?

<https://milvus.io/ai-quick-reference/what-are-emotional-ai-agents>

2 5 12 13 SmythOS - How Emotion Recognition is Transforming Conversational Agents

<https://smythos.com/developers/agent-development/conversational-agents-and-emotion-recognition/>

3 Model context protocol (MCP) - OpenAI Agents SDK

<https://openai.github.io/openai-agents-python/mcp/>

4 7 10 11 Role-based AI Agents | HabiliAI Docs

<https://docs.habili.ai/Intro/Role-based%20AI%20Agents>

6 Emotion Markup Language - Wikipedia

[https://en.wikipedia.org/wiki/Emotion\\_Markup\\_Language](https://en.wikipedia.org/wiki/Emotion_Markup_Language)

8 Web API Design Best Practices - Azure Architecture Center | Microsoft Learn

<https://learn.microsoft.com/en-us/azure/architecture/best-practices/api-design>

9 AI SDK by Vercel

<https://ai-sdk.dev/docs/introduction>